

Практикум №1-2

Створення процедур завантаження даних

Виконав студент 2 курсу Янов Богдан

1. Опис джерел даних

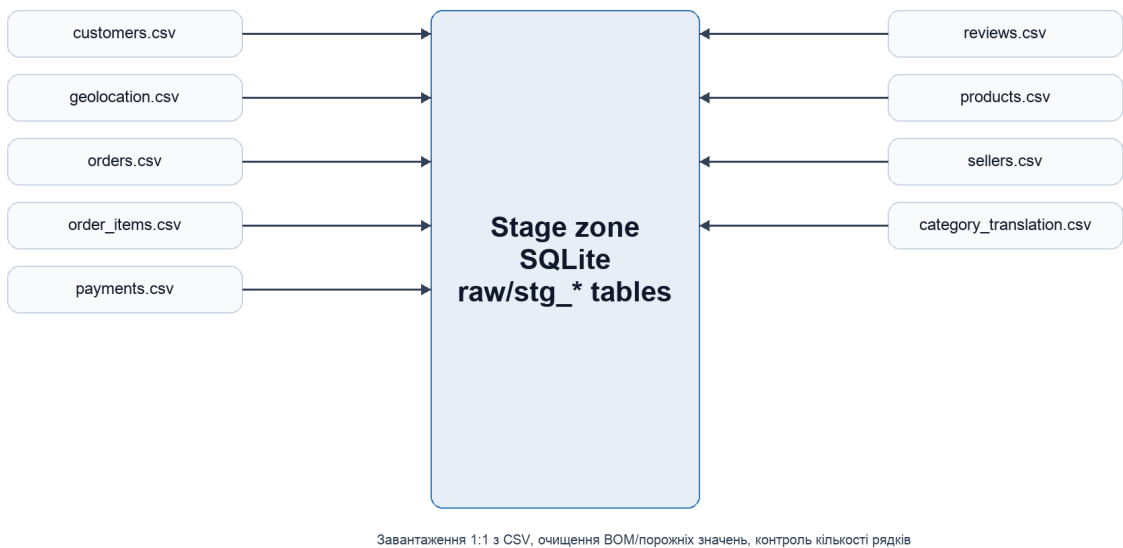
Для лабораторних робіт використано класичний набір відкритих даних Olist. Джерела мають спільні виміри: замовлення, товари, покупці, продавці, категорії, дати та локації.

Файл	Stage-таблиця	Призначення	Рядків
olist_customers_dataset.csv	stg_customers	Вимір покупців і локацій.	99,441
olist_geolocation_dataset.csv	stg_geolocation	Збагачення локацій координатами.	1,000,163
olist_orders_dataset.csv	stg_orders	Статуси та дати замовлень.	99,441
olist_order_items_dataset.csv	stg_order_items	Факт продажів на рівні позиції.	112,650
olist_order_payments_dataset.csv	stg_order_payments	Факт оплат.	103,886
olist_order_reviews_dataset.csv	stg_order_reviews	Факт відгуків.	99,224
olist_products_dataset.csv	stg_products	Вимір товарів.	32,951
olist_sellers_dataset.csv	stg_sellers	Вимір продавців.	3,095
product_category_name_translation.csv	stg_product_category_translation	Вимір категорій.	71

2. Модель Stage-зони

Stage-зона зберігає сирі дані 1:1 із CSV. На цьому етапі виконується технічне очищення: зняття BOM, перетворення порожніх рядків на NULL, фіксація кількості рядків і часу завантаження.

Модель Stage-зони Olist

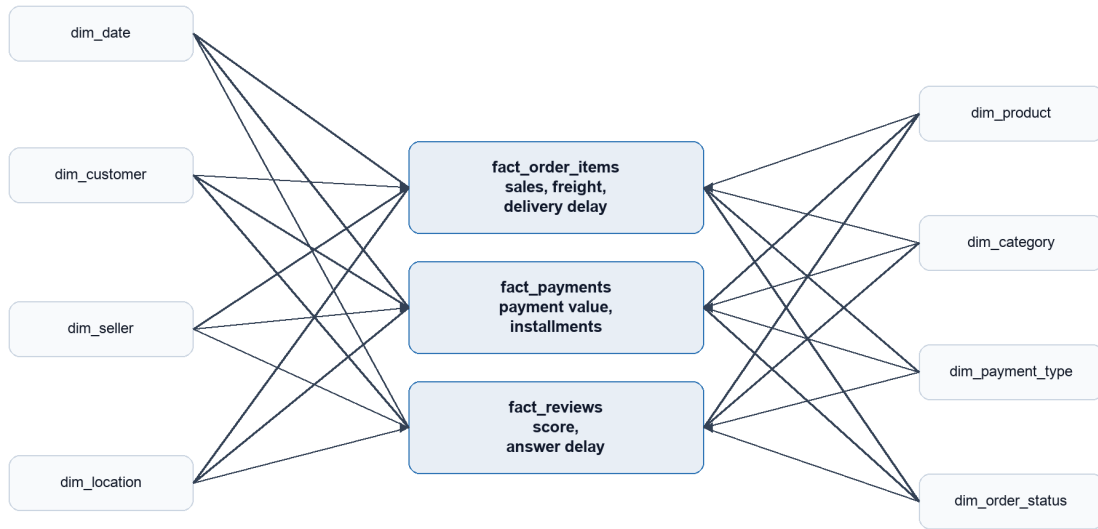


Таблиця	Опис	Рядків
stg_customers	Покупці: ідентифікатори клієнтів, місто, штат, ZIP-префікс.	99,441
stg_geolocation	Геолокація ZIP-префіксів: координати, місто та штат.	1,000,163
stg_orders	Замовлення: статуси та часові мітки життєвого циклу.	99,441
stg_order_items	Позиції замовлень: товар, продавець, ціна, доставка.	112,650
stg_order_payments	Оплати: тип платежу, кількість платежів, сума.	103,886
stg_order_reviews	Відгуки: оцінка, коментарі, дата створення і відповіді.	99,224
stg_products	Товари: категорія, габарити, вага, довжини описів.	32,951
stg_sellers	Продавці: ідентифікатор, місто, штат, ZIP-префікс.	3,095
stg_product_category_translation	Переклад категорій товарів з португальської на англійську.	71

3. Модель основного сховища

Основне сховище спроектовано як сузір'я фактів. Три фактові таблиці використовують конформні виміри для аналізу продажів, оплат, відгуків, доставки, географії та товарних категорій.

Модель основного сховища: сузір'я фактів



3.1. Виміри

Таблиця	Опис	Рядків
dim_date	Календарний вимір для дат купівлі, доставки, відгуків та лімітів відправлення.	755
dim_location	Вимір локацій за ZIP-префіксом, містом, штатом і середніми координатами.	28,220
dim_customer	Вимір покупців із посиланням на локацію.	99,441
dim_seller	Вимір продавців із посиланням на локацію.	3,095
dim_product	Вимір товарів із фізичними характеристиками та категорією.	32,951
dim_category	Вимір категорій з локальною та англомовною назвою.	74
dim_payment_type	Вимір способів оплати.	5
dim_order_status	Вимір статусів замовлення.	8

3.2. Таблиці фактів

Таблиця	Опис	Рядків
fact_order_items	Факт продажів на рівні позиції замовлення: ціна, доставка, затримка.	112,650
fact_payments	Факт оплат на рівні платежу: сума, тип, кількість частин.	103,886

fact_reviews	Факт відгуків: оцінка, наявність коментаря, час відповіді.	99,224
--------------	--	--------

4. ETL-засоби

- scripts/etl_olist.py - керує повним процесом Extract, Transform, Load.
- sql/01_stage_schema.sql - створює stage-таблиці.
- sql/02_warehouse_schema.sql - створює виміри, факти, ключі та індекси.
- sql/03_transform_load.sql - виконує нормалізацію, розрахунки і UPSERT.

Можливість завантаження змінених і додаткових даних реалізована через природні ключі та операції ON CONFLICT DO UPDATE/DO NOTHING. Повторний запуск ETL не дублює записи, а оновлює атрибути вимірів і фактів, якщо відповідні CSV змінилися.

1. Extract: читання дев'яти CSV-файлів у stage-таблиці.
2. Transform: очищення тексту, нормалізація статусів, категорій, локацій і дат.
3. Load: наповнення вимірів, потім фактів із зовнішніми ключами.
4. Validate: контроль кількості рядків і ключових помилок якості.

5. Результати завантаження

Останній запуск ETL: run_id=2, тривалість 8.99 с. Повторний запуск без --reset підтвердив ідемпотентність: кількості рядків у warehouse не збільшилися.

Перевірка	Результат
fact_order_items_without_customer	0
fact_order_items_without_product	0
fact_order_items_without_seller	0
fact_payments_without_payment_type	0
fact_reviews_out_of_range	0

5.1. Топ категорій за виручкою

Категорія	Позицій	Виручка	Середня оцінка
health_beauty	9,727	1,446,622.08	4.14
watches_gifts	6,001	1,306,761.40	4.02
bed_bath_table	11,270	1,258,189.51	3.90
sports_leisure	8,700	1,163,329.98	4.11
computers_accessories	7,894	1,068,070.48	3.93
furniture_decor	8,415	910,683.05	3.90
housewares	6,989	781,170.03	4.06

cool_stuff	3,806	721,492.90	4.15
auto	4,256	687,374.19	4.07
garden_tools	4,361	585,646.54	4.04

6. Висновок

У роботі спроектовано stage-зону та основне сховище для Olist, створено ETL-скрипти, виконано повне завантаження даних і перевірено повторне ідемпотентне завантаження. Сховище містить понад п'ять вимірів і три фактові таблиці, що достатньо для подальшої курсової роботи.

```

# etl_olist.py

import argparse
import csv
import json
import sqlite3
import sys
import time

from datetime import UTC, datetime
from pathlib import Path

ROOT_DIR = Path(__file__).resolve().parents[2]
LAB_DIR = Path(__file__).resolve().parents[1]
SQL_DIR = LAB_DIR / "sql"
DEFAULT_DATA_DIR = ROOT_DIR / "data"
DEFAULT_DB_PATH = LAB_DIR / "output" / "olist_dw.sqlite"
DEFAULT_SUMMARY_PATH = LAB_DIR / "output" / "etl_run_summary.json"

DATASETS = [
    ("olist_customers_dataset.csv", "stg_customers"),
    ("olist_geolocation_dataset.csv", "stg_geolocation"),
    ("olist_orders_dataset.csv", "stg_orders"),
    ("olist_order_items_dataset.csv", "stg_order_items"),
    ("olist_order_payments_dataset.csv", "stg_order_payments"),
    ("olist_order_reviews_dataset.csv", "stg_order_reviews"),
    ("olist_products_dataset.csv", "stg_products"),
    ("olist_sellers_dataset.csv", "stg_sellers"),
    ("product_category_name_translation.csv", "stg_product_category_translation"),
]

WAREHOUSE_TABLES = [
    "dim_date",
    "dim_location",
    "dim_order_status",
    "dim_payment_type",

```

```

    "dim_category",
    "dim_product",
    "dim_customer",
    "dim_seller",
    "fact_order_items",
    "fact_payments",
    "fact_reviews",
]

def read_sql(name: str) -> str:
    return (SQL_DIR / name).read_text(encoding="utf-8")

def connect(db_path: Path) -> sqlite3.Connection:
    db_path.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(db_path)
    conn.execute("PRAGMA foreign_keys = ON")
    conn.execute("PRAGMA journal_mode = WAL")
    conn.execute("PRAGMA synchronous = NORMAL")
    conn.execute("PRAGMA temp_store = MEMORY")
    return conn

def execute_schema(conn: sqlite3.Connection) -> None:
    conn.executescript(read_sql("01_stage_schema.sql"))
    conn.executescript(read_sql("02_warehouse_schema.sql"))
    conn.commit()

def normalize_row(row: dict[str, str]) -> dict[str, object]:
    normalized: dict[str, object] = {}
    for key, value in row.items():
        clean_key = key.lstrip("\ufeff")
        if value == "":
            normalized[clean_key] = None
        else:

```

```

        normalized[clean_key] = value
    return normalized

def truncate_stage(conn: sqlite3.Connection) -> None:
    for _, table in DATASETS:
        conn.execute(f"DELETE FROM {table}")
    conn.commit()

def load_csv_to_stage(conn: sqlite3.Connection, data_dir: Path) -> dict[str, int]:
    truncate_stage(conn)
    loaded_counts: dict[str, int] = {}
    now = datetime.now(UTC).isoformat(timespec="seconds").replace("+00:00", "Z")

    for file_name, table in DATASETS:
        path = data_dir / file_name
        if not path.exists():
            raise FileNotFoundError(f"Missing source file: {path}")

        with path.open("r", encoding="utf-8-sig", newline="") as source:
            reader = csv.DictReader(source)
            if not reader.fieldnames:
                raise ValueError(f"CSV file has no header: {path}")

            columns = [column.lstrip("\ufeff") for column in reader.fieldnames]
            placeholders = ", ".join(["?"] * len(columns))
            column_sql = ", ".join(columns)
            insert_sql = f"INSERT OR REPLACE INTO {table} ({column_sql}) VALUES ({placeholders})"

            batch: list[tuple[object, ...]] = []
            count = 0
            for row in reader:
                normalized = normalize_row(row)
                batch.append(tuple(normalized.get(column) for column in columns))

```

```

        if len(batch) >= 5000:
            conn.executemany(insert_sql, batch)
            count += len(batch)
            batch.clear()
    if batch:
        conn.executemany(insert_sql, batch)
        count += len(batch)

conn.execute(
    """
    INSERT INTO etl_file_log (file_name, loaded_at, row_count, source_mtime)
    VALUES (?, ?, ?, ?)
    ON CONFLICT (file_name) DO UPDATE SET
        loaded_at = excluded.loaded_at,
        row_count = excluded.row_count,
        source_mtime = excluded.source_mtime
    """,
    (file_name, now, count, path.stat().st_mtime),
)
loaded_counts[table] = count
conn.commit()

return loaded_counts


def transform_to_warehouse(conn: sqlite3.Connection) -> None:
    conn.executescript(read_sql("03_transform_load.sql"))
    conn.commit()


def count_rows(conn: sqlite3.Connection, tables: list[str]) -> dict[str, int]:
    return {
        table: int(conn.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0])
        for table in tables
    }

```

```

def run_quality_checks(conn: sqlite3.Connection) -> dict[str, int]:
    checks = {
        "fact_order_items_without_customer": """
            SELECT COUNT(*) FROM fact_order_items WHERE customer_key IS NULL
        """,
        "fact_order_items_without_product": """
            SELECT COUNT(*) FROM fact_order_items WHERE product_key IS NULL
        """,
        "fact_order_items_without_seller": """
            SELECT COUNT(*) FROM fact_order_items WHERE seller_key IS NULL
        """,
        "fact_payments_without_payment_type": """
            SELECT COUNT(*) FROM fact_payments WHERE payment_type_key IS NULL
        """,
        "fact_reviews_out_of_range": """
            SELECT COUNT(*) FROM fact_reviews WHERE review_score < 1 OR review_score >
5
            """,
    }

    return {name: int(conn.execute(sql).fetchone()[0]) for name, sql in
checks.items()}

```

```

def top_category_metrics(conn: sqlite3.Connection) -> list[dict[str, object]]:
    rows = conn.execute(
        """
        SELECT
            c.category_name_english,
            COUNT(*) AS order_items,
            ROUND(SUM(f.total_item_value), 2) AS total_revenue,
            ROUND(AVG(r.review_score), 2) AS avg_review_score
        FROM fact_order_items f
        JOIN dim_product p ON p.product_key = f.product_key
        JOIN dim_category c ON c.category_key = p.category_key
        LEFT JOIN fact_reviews r ON r.order_id = f.order_id
        GROUP BY c.category_name_english
        """
    )

```

```

        ORDER BY total_revenue DESC
        LIMIT 10
        """
    ).fetchall()
    return [
        {
            "category": row[0],
            "order_items": row[1],
            "total_revenue": row[2],
            "avg_review_score": row[3],
        }
        for row in rows
    ]

def run_etl(data_dir: Path, db_path: Path, reset: bool) -> dict[str, object]:
    if reset and db_path.exists():
        db_path.unlink()
        wal = db_path.with_suffix(db_path.suffix + "-wal")
        shm = db_path.with_suffix(db_path.suffix + "-shm")
        for sidecar in (wal, shm):
            if sidecar.exists():
                sidecar.unlink()

    started_at = datetime.now(UTC).isoformat(timespec="seconds").replace("+00:00",
"Z")
    start_time = time.perf_counter()

    with connect(db_path) as conn:
        execute_schema(conn)
        conn.execute(
            "INSERT INTO etl_run_log (started_at, status, message) VALUES (?, ?, ?)",
            (started_at, "running", "Stage and warehouse load started"),
        )
        run_id = int(conn.execute("SELECT last_insert_rowid()").fetchone()[0])
        conn.commit()

```

```

try:
    stage_counts = load_csv_to_stage(conn, data_dir)
    transform_to_warehouse(conn)
    warehouse_counts = count_rows(conn, WAREHOUSE_TABLES)
    quality_checks = run_quality_checks(conn)
    stage_total = sum(stage_counts.values())
    warehouse_total = sum(warehouse_counts.values())
    finished_at =
datetime.now(UTC).isoformat(timespec="seconds").replace("+00:00", "Z")
    conn.execute(
        """
        UPDATE etl_run_log
        SET finished_at = ?, status = ?, stage_rows = ?, warehouse_rows = ?,
message = ?
        WHERE run_id = ?
        """,
        (finished_at, "success", stage_total, warehouse_total, "ETL completed",
run_id),
    )
    conn.commit()
except Exception as exc:
    finished_at =
datetime.now(UTC).isoformat(timespec="seconds").replace("+00:00", "Z")
    conn.execute(
        """
        UPDATE etl_run_log
        SET finished_at = ?, status = ?, message = ?
        WHERE run_id = ?
        """,
        (finished_at, "failed", str(exc), run_id),
    )
    conn.commit()
    raise

return {

```

```

        "run_id": run_id,
        "started_at": started_at,
        "finished_at": finished_at,
        "duration_seconds": round(time.perf_counter() - start_time, 2),
        "database": str(db_path),
        "stage_counts": stage_counts,
        "warehouse_counts": warehouse_counts,
        "quality_checks": quality_checks,
        "top_category_metrics": top_category_metrics(conn),
    }

```

```

def parse_args(argv: list[str]) -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Load Olist CSV data into a
dimensional SQLite warehouse.")
    parser.add_argument("--data-dir", type=Path, default=DEFAULT_DATA_DIR,
help="Directory with Olist CSV files.")
    parser.add_argument("--db", type=Path, default=DEFAULT_DB_PATH, help="Output
SQLite database path.")
    parser.add_argument("--summary", type=Path, default=DEFAULT_SUMMARY_PATH,
help="Where to write JSON run summary.")
    parser.add_argument("--reset", action="store_true", help="Delete the existing
database before loading.")
    return parser.parse_args(argv)

```

```

def main() -> int:
    args = parse_args(sys.argv[1:])
    summary = run_etl(args.data_dir.resolve(), args.db.resolve(), args.reset)
    args.summary.parent.mkdir(parents=True, exist_ok=True)
    args.summary.write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding="utf-8")
    print(json.dumps(summary, ensure_ascii=False, indent=2))

```

```

if __name__ == "__main__":
    main()

```

```
-- 01_stage_schema.sql
```

```
CREATE TABLE IF NOT EXISTS stg_customers (  
    customer_id TEXT PRIMARY KEY,  
    customer_unique_id TEXT,  
    customer_zip_code_prefix TEXT,  
    customer_city TEXT,  
    customer_state TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS stg_geolocation (  
    geolocation_zip_code_prefix TEXT,  
    geolocation_lat REAL,  
    geolocation_lng REAL,  
    geolocation_city TEXT,  
    geolocation_state TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS stg_orders (  
    order_id TEXT PRIMARY KEY,  
    customer_id TEXT,  
    order_status TEXT,  
    order_purchase_timestamp TEXT,  
    order_approved_at TEXT,  
    order_delivered_carrier_date TEXT,  
    order_delivered_customer_date TEXT,  
    order_estimated_delivery_date TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS stg_order_items (  
    order_id TEXT,  
    order_item_id INTEGER,  
    product_id TEXT,  
    seller_id TEXT,  
    shipping_limit_date TEXT,
```

```

    price REAL,
    freight_value REAL,
    PRIMARY KEY (order_id, order_item_id)
);

CREATE TABLE IF NOT EXISTS stg_order_payments (
    order_id TEXT,
    payment_sequential INTEGER,
    payment_type TEXT,
    payment_installments INTEGER,
    payment_value REAL,
    PRIMARY KEY (order_id, payment_sequential)
);

CREATE TABLE IF NOT EXISTS stg_order_reviews (
    review_id TEXT,
    order_id TEXT,
    review_score INTEGER,
    review_comment_title TEXT,
    review_comment_message TEXT,
    review_creation_date TEXT,
    review_answer_timestamp TEXT,
    PRIMARY KEY (review_id, order_id)
);

CREATE TABLE IF NOT EXISTS stg_products (
    product_id TEXT PRIMARY KEY,
    product_category_name TEXT,
    product_name_lenght INTEGER,
    product_description_lenght INTEGER,
    product_photos_qty INTEGER,
    product_weight_g INTEGER,
    product_length_cm INTEGER,
    product_height_cm INTEGER,
    product_width_cm INTEGER
);

```

```
CREATE TABLE IF NOT EXISTS stg_sellers (  
    seller_id TEXT PRIMARY KEY,  
    seller_zip_code_prefix TEXT,  
    seller_city TEXT,  
    seller_state TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS stg_product_category_translation (  
    product_category_name TEXT PRIMARY KEY,  
    product_category_name_english TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS etl_file_log (  
    file_name TEXT PRIMARY KEY,  
    loaded_at TEXT NOT NULL,  
    row_count INTEGER NOT NULL,  
    source_mtime REAL NOT NULL  
);
```

```
-- 02_warehouse_schema.sql
```

```
CREATE TABLE IF NOT EXISTS dim_date (  
    date_key INTEGER PRIMARY KEY,  
    full_date TEXT NOT NULL UNIQUE,  
    year INTEGER NOT NULL,  
    quarter INTEGER NOT NULL,  
    month INTEGER NOT NULL,  
    month_name TEXT NOT NULL,  
    day INTEGER NOT NULL,  
    day_of_week INTEGER NOT NULL,  
    day_name TEXT NOT NULL,  
    week_of_year INTEGER NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS dim_location (  
    location_key INTEGER PRIMARY KEY AUTOINCREMENT,  
    zip_code_prefix TEXT NOT NULL,  
    city TEXT NOT NULL,  
    state TEXT NOT NULL,  
    avg_lat REAL,  
    avg_lng REAL,  
    source_count INTEGER NOT NULL DEFAULT 0,  
    UNIQUE (zip_code_prefix, city, state)  
);
```

```
CREATE TABLE IF NOT EXISTS dim_order_status (  
    status_key INTEGER PRIMARY KEY AUTOINCREMENT,  
    order_status TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS dim_payment_type (  
    payment_type_key INTEGER PRIMARY KEY AUTOINCREMENT,  
    payment_type TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS dim_category (  
    category_key INTEGER PRIMARY KEY AUTOINCREMENT,  
    category_name TEXT NOT NULL UNIQUE,  
    category_name_english TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS dim_product (  
    product_key INTEGER PRIMARY KEY AUTOINCREMENT,  
    product_id TEXT NOT NULL UNIQUE,  
    category_key INTEGER,  
    product_name_length INTEGER,  
    product_description_length INTEGER,  
    product_photos_qty INTEGER,  
    product_weight_g INTEGER,  
    product_length_cm INTEGER,
```

```
product_height_cm INTEGER,  
product_width_cm INTEGER,  
product_hash TEXT NOT NULL,  
updated_at TEXT NOT NULL,  
FOREIGN KEY (category_key) REFERENCES dim_category(category_key)  
);
```

```
CREATE TABLE IF NOT EXISTS dim_customer (  
    customer_key INTEGER PRIMARY KEY AUTOINCREMENT,  
    customer_id TEXT NOT NULL UNIQUE,  
    customer_unique_id TEXT NOT NULL,  
    location_key INTEGER,  
    zip_code_prefix TEXT NOT NULL,  
    city TEXT NOT NULL,  
    state TEXT NOT NULL,  
    updated_at TEXT NOT NULL,  
    FOREIGN KEY (location_key) REFERENCES dim_location(location_key)  
);
```

```
CREATE TABLE IF NOT EXISTS dim_seller (  
    seller_key INTEGER PRIMARY KEY AUTOINCREMENT,  
    seller_id TEXT NOT NULL UNIQUE,  
    location_key INTEGER,  
    zip_code_prefix TEXT NOT NULL,  
    city TEXT NOT NULL,  
    state TEXT NOT NULL,  
    updated_at TEXT NOT NULL,  
    FOREIGN KEY (location_key) REFERENCES dim_location(location_key)  
);
```

```
CREATE TABLE IF NOT EXISTS fact_order_items (  
    order_item_fact_key INTEGER PRIMARY KEY AUTOINCREMENT,  
    order_id TEXT NOT NULL,  
    order_item_id INTEGER NOT NULL,  
    purchase_date_key INTEGER,  
    shipping_limit_date_key INTEGER,
```

```

customer_key INTEGER,
seller_key INTEGER,
product_key INTEGER,
customer_location_key INTEGER,
seller_location_key INTEGER,
status_key INTEGER,
price REAL NOT NULL,
freight_value REAL NOT NULL,
total_item_value REAL NOT NULL,
days_to_delivery REAL,
delivery_delay_days REAL,
was_delivered_late INTEGER,
row_hash TEXT NOT NULL,
updated_at TEXT NOT NULL,
UNIQUE (order_id, order_item_id),
FOREIGN KEY (purchase_date_key) REFERENCES dim_date(date_key),
FOREIGN KEY (shipping_limit_date_key) REFERENCES dim_date(date_key),
FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key),
FOREIGN KEY (seller_key) REFERENCES dim_seller(seller_key),
FOREIGN KEY (product_key) REFERENCES dim_product(product_key),
FOREIGN KEY (customer_location_key) REFERENCES dim_location(location_key),
FOREIGN KEY (seller_location_key) REFERENCES dim_location(location_key),
FOREIGN KEY (status_key) REFERENCES dim_order_status(status_key)
);

```

```

CREATE TABLE IF NOT EXISTS fact_payments (
    payment_fact_key INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id TEXT NOT NULL,
    payment_sequential INTEGER NOT NULL,
    purchase_date_key INTEGER,
    customer_key INTEGER,
    payment_type_key INTEGER,
    status_key INTEGER,
    payment_installments INTEGER NOT NULL,
    payment_value REAL NOT NULL,
    row_hash TEXT NOT NULL,

```

```

updated_at TEXT NOT NULL,
UNIQUE (order_id, payment_sequential),
FOREIGN KEY (purchase_date_key) REFERENCES dim_date(date_key),
FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key),
FOREIGN KEY (payment_type_key) REFERENCES dim_payment_type(payment_type_key),
FOREIGN KEY (status_key) REFERENCES dim_order_status(status_key)
);

```

```

CREATE TABLE IF NOT EXISTS fact_reviews (
    review_fact_key INTEGER PRIMARY KEY AUTOINCREMENT,
    review_id TEXT NOT NULL,
    order_id TEXT NOT NULL,
    review_date_key INTEGER,
    customer_key INTEGER,
    status_key INTEGER,
    review_score INTEGER NOT NULL,
    has_comment INTEGER NOT NULL,
    answer_delay_hours REAL,
    row_hash TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    UNIQUE (review_id, order_id),
    FOREIGN KEY (review_date_key) REFERENCES dim_date(date_key),
    FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key),
    FOREIGN KEY (status_key) REFERENCES dim_order_status(status_key)
);

```

```

CREATE TABLE IF NOT EXISTS etl_run_log (
    run_id INTEGER PRIMARY KEY AUTOINCREMENT,
    started_at TEXT NOT NULL,
    finished_at TEXT,
    status TEXT NOT NULL,
    stage_rows INTEGER DEFAULT 0,
    warehouse_rows INTEGER DEFAULT 0,
    message TEXT
);

```

```

CREATE INDEX IF NOT EXISTS idx_dim_location_nk ON dim_location(zip_code_prefix, city,
state);
CREATE INDEX IF NOT EXISTS idx_dim_customer_location ON dim_customer(location_key);
CREATE INDEX IF NOT EXISTS idx_dim_seller_location ON dim_seller(location_key);
CREATE INDEX IF NOT EXISTS idx_fact_order_items_date ON
fact_order_items(purchase_date_key);
CREATE INDEX IF NOT EXISTS idx_fact_order_items_customer ON
fact_order_items(customer_key);
CREATE INDEX IF NOT EXISTS idx_fact_order_items_seller ON
fact_order_items(seller_key);
CREATE INDEX IF NOT EXISTS idx_fact_order_items_product ON
fact_order_items(product_key);
CREATE INDEX IF NOT EXISTS idx_fact_payments_date ON fact_payments(purchase_date_key);
CREATE INDEX IF NOT EXISTS idx_fact_reviews_date ON fact_reviews(review_date_key);

```

```
-- 03_transform_load.sql
```

```
PRAGMA foreign_keys = ON;
```

```

INSERT OR IGNORE INTO dim_date (
    date_key, full_date, year, quarter, month, month_name, day, day_of_week, day_name,
    week_of_year
)
WITH raw_dates(full_date) AS (
    SELECT substr(order_purchase_timestamp, 1, 10) FROM stg_orders WHERE
order_purchase_timestamp IS NOT NULL AND order_purchase_timestamp <> ''
    UNION
    SELECT substr(order_approved_at, 1, 10) FROM stg_orders WHERE order_approved_at IS
NOT NULL AND order_approved_at <> ''
    UNION
    SELECT substr(order_delivered_carrier_date, 1, 10) FROM stg_orders WHERE
order_delivered_carrier_date IS NOT NULL AND order_delivered_carrier_date <> ''
    UNION
    SELECT substr(order_delivered_customer_date, 1, 10) FROM stg_orders WHERE
order_delivered_customer_date IS NOT NULL AND order_delivered_customer_date <> ''

```

```

UNION
SELECT substr(order_estimated_delivery_date, 1, 10) FROM stg_orders WHERE
order_estimated_delivery_date IS NOT NULL AND order_estimated_delivery_date <> ''
UNION
SELECT substr(shipping_limit_date, 1, 10) FROM stg_order_items WHERE
shipping_limit_date IS NOT NULL AND shipping_limit_date <> ''
UNION
SELECT substr(review_creation_date, 1, 10) FROM stg_order_reviews WHERE
review_creation_date IS NOT NULL AND review_creation_date <> ''
UNION
SELECT substr(review_answer_timestamp, 1, 10) FROM stg_order_reviews WHERE
review_answer_timestamp IS NOT NULL AND review_answer_timestamp <> ''
),
valid_dates AS (
SELECT DISTINCT full_date
FROM raw_dates
WHERE full_date GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]'
)
SELECT
CAST(strftime('%Y%m%d', full_date) AS INTEGER) AS date_key,
full_date,
CAST(strftime('%Y', full_date) AS INTEGER) AS year,
((CAST(strftime('%m', full_date) AS INTEGER) - 1) / 3) + 1 AS quarter,
CAST(strftime('%m', full_date) AS INTEGER) AS month,
CASE strftime('%m', full_date)
    WHEN '01' THEN 'January' WHEN '02' THEN 'February' WHEN '03' THEN 'March'
    WHEN '04' THEN 'April' WHEN '05' THEN 'May' WHEN '06' THEN 'June'
    WHEN '07' THEN 'July' WHEN '08' THEN 'August' WHEN '09' THEN 'September'
    WHEN '10' THEN 'October' WHEN '11' THEN 'November' ELSE 'December'
END AS month_name,
CAST(strftime('%d', full_date) AS INTEGER) AS day,
CAST(strftime('%w', full_date) AS INTEGER) + 1 AS day_of_week,
CASE strftime('%w', full_date)
    WHEN '0' THEN 'Sunday' WHEN '1' THEN 'Monday' WHEN '2' THEN 'Tuesday'
    WHEN '3' THEN 'Wednesday' WHEN '4' THEN 'Thursday' WHEN '5' THEN 'Friday'
    ELSE 'Saturday'

```

```

        END AS day_name,
        CAST(strftime('%W', full_date) AS INTEGER) AS week_of_year
FROM valid_dates;

INSERT INTO dim_location (zip_code_prefix, city, state, avg_lat, avg_lng,
source_count)
SELECT
    geolocation_zip_code_prefix,
    lower(trim(geolocation_city)),
    upper(trim(geolocation_state)),
    avg(geolocation_lat),
    avg(geolocation_lng),
    count(*)
FROM stg_geolocation
WHERE geolocation_zip_code_prefix IS NOT NULL
    AND geolocation_zip_code_prefix <> ''
    AND geolocation_city IS NOT NULL
    AND geolocation_state IS NOT NULL
GROUP BY geolocation_zip_code_prefix, lower(trim(geolocation_city)),
upper(trim(geolocation_state))
ON CONFLICT (zip_code_prefix, city, state) DO UPDATE SET
    avg_lat = excluded.avg_lat,
    avg_lng = excluded.avg_lng,
    source_count = excluded.source_count;

INSERT INTO dim_location (zip_code_prefix, city, state, avg_lat, avg_lng,
source_count)
SELECT DISTINCT
    customer_zip_code_prefix,
    lower(trim(customer_city)),
    upper(trim(customer_state)),
    NULL,
    NULL,
    0
FROM stg_customers
WHERE customer_zip_code_prefix IS NOT NULL

```

```

    AND customer_city IS NOT NULL
    AND customer_state IS NOT NULL
ON CONFLICT (zip_code_prefix, city, state) DO NOTHING;

INSERT INTO dim_location (zip_code_prefix, city, state, avg_lat, avg_lng,
source_count)
SELECT DISTINCT
    seller_zip_code_prefix,
    lower(trim(seller_city)),
    upper(trim(seller_state)),
    NULL,
    NULL,
    0
FROM stg_sellers
WHERE seller_zip_code_prefix IS NOT NULL
    AND seller_city IS NOT NULL
    AND seller_state IS NOT NULL
ON CONFLICT (zip_code_prefix, city, state) DO NOTHING;

INSERT INTO dim_order_status (order_status)
SELECT DISTINCT lower(trim(order_status))
FROM stg_orders
WHERE order_status IS NOT NULL AND trim(order_status) <> ''
ON CONFLICT (order_status) DO NOTHING;

INSERT INTO dim_payment_type (payment_type)
SELECT DISTINCT lower(trim(payment_type))
FROM stg_order_payments
WHERE payment_type IS NOT NULL AND trim(payment_type) <> ''
ON CONFLICT (payment_type) DO NOTHING;

INSERT INTO dim_category (category_name, category_name_english)
SELECT
    category_name,
    COALESCE(NULLIF(trim(t.product_category_name_english), ''), category_name) AS
category_name_english

```

```

FROM (
    SELECT DISTINCT COALESCE(NULLIF(trim(product_category_name), ''), 'unknown') AS
category_name
    FROM stg_products
    UNION
    SELECT DISTINCT COALESCE(NULLIF(trim(product_category_name), ''), 'unknown') AS
category_name
    FROM stg_product_category_translation
) c
LEFT JOIN stg_product_category_translation t
    ON t.product_category_name = c.category_name
ON CONFLICT (category_name) DO UPDATE SET
    category_name_english = excluded.category_name_english;

INSERT INTO dim_product (
    product_id, category_key, product_name_length, product_description_length,
    product_photos_qty, product_weight_g, product_length_cm, product_height_cm,
    product_width_cm, product_hash, updated_at
)
SELECT
    p.product_id,
    c.category_key,
    p.product_name_lenght,
    p.product_description_lenght,
    p.product_photos_qty,
    p.product_weight_g,
    p.product_length_cm,
    p.product_height_cm,
    p.product_width_cm,
    p.product_id || '|' ||
        COALESCE(p.product_category_name, 'unknown') || '|' ||
        COALESCE(CAST(p.product_name_lenght AS TEXT), '') || '|' ||
        COALESCE(CAST(p.product_description_lenght AS TEXT), '') || '|' ||
        COALESCE(CAST(p.product_photos_qty AS TEXT), '') || '|' ||
        COALESCE(CAST(p.product_weight_g AS TEXT), '') || '|' ||
        COALESCE(CAST(p.product_length_cm AS TEXT), '') || '|' ||

```

```

        COALESCE(CAST(p.product_height_cm AS TEXT), '') || '|' ||
        COALESCE(CAST(p.product_width_cm AS TEXT), '') AS product_hash,
        datetime('now')
FROM stg_products p
LEFT JOIN dim_category c
    ON c.category_name = COALESCE(NULLIF(trim(p.product_category_name), ''),
'unknown')
ON CONFLICT (product_id) DO UPDATE SET
    category_key = excluded.category_key,
    product_name_length = excluded.product_name_length,
    product_description_length = excluded.product_description_length,
    product_photos_qty = excluded.product_photos_qty,
    product_weight_g = excluded.product_weight_g,
    product_length_cm = excluded.product_length_cm,
    product_height_cm = excluded.product_height_cm,
    product_width_cm = excluded.product_width_cm,
    product_hash = excluded.product_hash,
    updated_at = excluded.updated_at
WHERE dim_product.product_hash <> excluded.product_hash;

INSERT INTO dim_customer (
    customer_id, customer_unique_id, location_key, zip_code_prefix, city, state,
    updated_at
)
SELECT
    c.customer_id,
    c.customer_unique_id,
    l.location_key,
    c.customer_zip_code_prefix,
    lower(trim(c.customer_city)),
    upper(trim(c.customer_state)),
    datetime('now')
FROM stg_customers c
LEFT JOIN dim_location l
    ON l.zip_code_prefix = c.customer_zip_code_prefix
    AND l.city = lower(trim(c.customer_city))

```

```

    AND l.state = upper(trim(c.customer_state))
ON CONFLICT (customer_id) DO UPDATE SET
    customer_unique_id = excluded.customer_unique_id,
    location_key = excluded.location_key,
    zip_code_prefix = excluded.zip_code_prefix,
    city = excluded.city,
    state = excluded.state,
    updated_at = excluded.updated_at
WHERE dim_customer.customer_unique_id <> excluded.customer_unique_id
    OR dim_customer.location_key IS NOT excluded.location_key
    OR dim_customer.zip_code_prefix <> excluded.zip_code_prefix
    OR dim_customer.city <> excluded.city
    OR dim_customer.state <> excluded.state;

INSERT INTO dim_seller (
    seller_id, location_key, zip_code_prefix, city, state, updated_at
)
SELECT
    s.seller_id,
    l.location_key,
    s.seller_zip_code_prefix,
    lower(trim(s.seller_city)),
    upper(trim(s.seller_state)),
    datetime('now')
FROM stg_sellers s
LEFT JOIN dim_location l
    ON l.zip_code_prefix = s.seller_zip_code_prefix
    AND l.city = lower(trim(s.seller_city))
    AND l.state = upper(trim(s.seller_state))
ON CONFLICT (seller_id) DO UPDATE SET
    location_key = excluded.location_key,
    zip_code_prefix = excluded.zip_code_prefix,
    city = excluded.city,
    state = excluded.state,
    updated_at = excluded.updated_at
WHERE dim_seller.location_key IS NOT excluded.location_key

```

```

OR dim_seller.zip_code_prefix <> excluded.zip_code_prefix
OR dim_seller.city <> excluded.city
OR dim_seller.state <> excluded.state;

INSERT INTO fact_order_items (
    order_id, order_item_id, purchase_date_key, shipping_limit_date_key,
    customer_key, seller_key, product_key, customer_location_key, seller_location_key,
    status_key, price, freight_value, total_item_value, days_to_delivery,
    delivery_delay_days, was_delivered_late, row_hash, updated_at
)
SELECT
    i.order_id,
    i.order_item_id,
    CAST(strftime('%Y%m%d', substr(o.order_purchase_timestamp, 1, 10)) AS INTEGER),
    CAST(strftime('%Y%m%d', substr(i.shipping_limit_date, 1, 10)) AS INTEGER),
    dc.customer_key,
    ds.seller_key,
    dp.product_key,
    dc.location_key,
    ds.location_key,
    dos.status_key,
    COALESCE(i.price, 0),
    COALESCE(i.freight_value, 0),
    COALESCE(i.price, 0) + COALESCE(i.freight_value, 0),
    CASE
        WHEN o.order_delivered_customer_date IS NULL OR
o.order_delivered_customer_date = '' THEN NULL
        ELSE round(julianday(o.order_delivered_customer_date) -
julianday(o.order_purchase_timestamp), 2)
    END,
    CASE
        WHEN o.order_delivered_customer_date IS NULL OR
o.order_delivered_customer_date = '' THEN NULL
        ELSE round(julianday(o.order_delivered_customer_date) -
julianday(o.order_estimated_delivery_date), 2)
    END,

```

```

CASE
    WHEN o.order_delivered_customer_date IS NULL OR
o.order_delivered_customer_date = '' THEN NULL
    WHEN julianday(o.order_delivered_customer_date) >
julianday(o.order_estimated_delivery_date) THEN 1
    ELSE 0
END,
i.order_id || '|' || i.order_item_id || '|' || COALESCE(i.product_id, '') || '|'
||
COALESCE(i.seller_id, '') || '|' || COALESCE(CAST(i.price AS TEXT), '') || '|'
||
COALESCE(CAST(i.freight_value AS TEXT), '') || '|' || COALESCE(o.order_status,
''),
datetime('now')
FROM stg_order_items i
JOIN stg_orders o ON o.order_id = i.order_id
LEFT JOIN dim_customer dc ON dc.customer_id = o.customer_id
LEFT JOIN dim_seller ds ON ds.seller_id = i.seller_id
LEFT JOIN dim_product dp ON dp.product_id = i.product_id
LEFT JOIN dim_order_status dos ON dos.order_status = lower(trim(o.order_status))
ON CONFLICT (order_id, order_item_id) DO UPDATE SET
    purchase_date_key = excluded.purchase_date_key,
    shipping_limit_date_key = excluded.shipping_limit_date_key,
    customer_key = excluded.customer_key,
    seller_key = excluded.seller_key,
    product_key = excluded.product_key,
    customer_location_key = excluded.customer_location_key,
    seller_location_key = excluded.seller_location_key,
    status_key = excluded.status_key,
    price = excluded.price,
    freight_value = excluded.freight_value,
    total_item_value = excluded.total_item_value,
    days_to_delivery = excluded.days_to_delivery,
    delivery_delay_days = excluded.delivery_delay_days,
    was_delivered_late = excluded.was_delivered_late,
    row_hash = excluded.row_hash,

```

```

        updated_at = excluded.updated_at
WHERE fact_order_items.row_hash <> excluded.row_hash;

INSERT INTO fact_payments (
    order_id, payment_sequential, purchase_date_key, customer_key,
    payment_type_key, status_key, payment_installments, payment_value,
    row_hash, updated_at
)
SELECT
    p.order_id,
    p.payment_sequential,
    CAST(strftime('%Y%m%d', substr(o.order_purchase_timestamp, 1, 10)) AS INTEGER),
    dc.customer_key,
    dpt.payment_type_key,
    dos.status_key,
    COALESCE(p.payment_installments, 0),
    COALESCE(p.payment_value, 0),
    p.order_id || '|' || p.payment_sequential || '|' || COALESCE(p.payment_type, '')
|| '|' ||
        COALESCE(CAST(p.payment_installments AS TEXT), '') || '|' ||
COALESCE(CAST(p.payment_value AS TEXT), ''),
    datetime('now')
FROM stg_order_payments p
JOIN stg_orders o ON o.order_id = p.order_id
LEFT JOIN dim_customer dc ON dc.customer_id = o.customer_id
LEFT JOIN dim_payment_type dpt ON dpt.payment_type = lower(trim(p.payment_type))
LEFT JOIN dim_order_status dos ON dos.order_status = lower(trim(o.order_status))
ON CONFLICT (order_id, payment_sequential) DO UPDATE SET
    purchase_date_key = excluded.purchase_date_key,
    customer_key = excluded.customer_key,
    payment_type_key = excluded.payment_type_key,
    status_key = excluded.status_key,
    payment_installments = excluded.payment_installments,
    payment_value = excluded.payment_value,
    row_hash = excluded.row_hash,
    updated_at = excluded.updated_at

```

```

WHERE fact_payments.row_hash <> excluded.row_hash;

INSERT INTO fact_reviews (
    review_id, order_id, review_date_key, customer_key, status_key,
    review_score, has_comment, answer_delay_hours, row_hash, updated_at
)
SELECT
    r.review_id,
    r.order_id,
    CAST(strftime('%Y%m%d', substr(r.review_creation_date, 1, 10)) AS INTEGER),
    dc.customer_key,
    dos.status_key,
    COALESCE(r.review_score, 0),
    CASE
        WHEN COALESCE(trim(r.review_comment_title), '') <> ''
            OR COALESCE(trim(r.review_comment_message), '') <> '' THEN 1
        ELSE 0
    END,
    CASE
        WHEN r.review_answer_timestamp IS NULL OR r.review_answer_timestamp = '' THEN
NULL
        ELSE round((julianday(r.review_answer_timestamp) -
julianday(r.review_creation_date)) * 24, 2)
    END,
    r.review_id || '|' || r.order_id || '|' || COALESCE(CAST(r.review_score AS TEXT),
    '') || '|' ||
        COALESCE(r.review_comment_title, '') || '|' ||
COALESCE(r.review_comment_message, ''),
    datetime('now')
FROM stg_order_reviews r
JOIN stg_orders o ON o.order_id = r.order_id
LEFT JOIN dim_customer dc ON dc.customer_id = o.customer_id
LEFT JOIN dim_order_status dos ON dos.order_status = lower(trim(o.order_status))
ON CONFLICT (review_id, order_id) DO UPDATE SET
    review_date_key = excluded.review_date_key,
    customer_key = excluded.customer_key,

```

```
status_key = excluded.status_key,  
review_score = excluded.review_score,  
has_comment = excluded.has_comment,  
answer_delay_hours = excluded.answer_delay_hours,  
row_hash = excluded.row_hash,  
updated_at = excluded.updated_at  
WHERE fact_reviews.row_hash <> excluded.row_hash;
```